Numerische Lineare Algebra: Iteration & Zufall

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- *Komplexitätsanalyse* / $O()$-Notation
- *Stabilität* und *Konditionierung*
- *Orthogonalmatrizen*
- *Spektralnorm* $\|\cdot\|_2$
- *LGS*: Problem, Direkte Algorithmen
- **Lineare Algebra:**
  - Eigenwerte und Eigenvektoren
  - Ähnlichkeitstransformationen: $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$
- **Wahrscheinlichkeitstheorie**: Zufallsvektoren
- **Analysis I**:
  - Konvergenz von Folgen
  - Geometrische Reihe $\sum_k p^k$ für $|p| < 1$

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ und $\boldsymbol{v} \neq \boldsymbol{0}$ ein Eigenvektor mit $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$. Welche Aussage gilt dann **immer**?

1. $\boldsymbol{A}^k \boldsymbol{v} = \lambda^k \boldsymbol{v}$ für alle $k \in \mathbb{N}$
2. Es muss $\|\boldsymbol{v}\| = 1$ gelten
3. $\lambda > 0$
4. $\boldsymbol{v}$ ist der einzige Eigenvektor zu λ

Frage 2: Sei $|p| < 1$. Was ist $\sum_{k=0}^{\infty} p^k$?

1. $\frac{1}{1-p}$
2. $\frac{p}{1-p}$
3. $\frac{1}{p-1}$
4. Die Reihe divergiert.

Überblick

- Numerische lineare Algebra ist ein sehr großes Feld.
- Wir haben bisher die wichtigsten Lösungsmethoden gesehen.
- Die Kernidee dabei war *Matrixzerlegung* (*LU*, *QR*, *Cholesky*, ...).
- Heute sehen wir in kürzerer Form zwei weitere wichtige Kernideen:
  - *Iterative Methoden*
    - verkürzen *Laufzeit* auf Kosten von Genauigkeit.
  - *Probabilistische Methoden*
    - ersetzen große Probleme durch kleinere mit *höchstwahrscheinlich* sehr ähnlicher Lösung.
- An beiden wird in der numerischen linearen Algebra aktiv geforscht.

Eigenwertprobleme

Lösungsverfahren für Eigenwertprobleme

- *Direkte Methoden existieren:*
  - Charakteristisches Polynom: $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0 \rightarrow$ Nullstellensuche
  - Problem:
    Polynom vom Grad n für $n \times n$ Matrix $\rightarrow$ keine direkte Lösung für $n \geq 5$
  - Numerisch instabil und teuer: $O(n^4)$ oder schlechter
- *Wann sind direkte Methoden unzureichend?*
  - Große Matrizen ($n > 1000$): $O(n^3)$ oder schlechter
  - Dünnbesetzte Matrizen: Direkte Methoden zerstören *sparsity*-Struktur
  - Oft brauchen wir nur wenige Eigenwerte (z.B. die größten $k \ll n$)
- *Warum iterativ?*
  - Schneller wenn nur wenige Eigenwerte benötigt
  - Nutzen *sparsity*-Struktur aus (nur Matrix-Vektor-Multiplikationen!)
  - Trade-off “Genauigkeit vs. Aufwand” ist steuerbar

Potenzmethode

Die **Potenzmethode** (nach Mises) liefert über Iteration den größten (separierten!) Eigenwert λ_1 und Eigenvektor $\boldsymbol{v}_1$ einer Matrix $\boldsymbol{A}$:

Idee:
Mit $\boldsymbol{x}^{(k)} = \frac{\boldsymbol{A}\boldsymbol{x}^{(k-1)}}{\left\|\boldsymbol{A}\boldsymbol{x}^{(k-1)}\right\|}$ gilt

$$\lambda^{(k)} = \left\|\boldsymbol{A}\boldsymbol{x}^{(k)}\right\| \xrightarrow{k\to\infty} |\lambda_1| \text{ und } \boldsymbol{x}^{(k)} \xrightarrow{k\to\infty} \pm\boldsymbol{v}_1$$

(Die Norm liefert nur den *Betrag* $|\lambda_1|$ – das Vorzeichen von λ_1 liefert sie nicht; für negatives λ_1 oszilliert $\boldsymbol{x}^{(k)}$ zwischen $\pm\boldsymbol{v}_1$.)

Intuition:

- Für diagonalisierbare $\boldsymbol{A}$ ist jeder Startvektor Linearkombination der Eigenvektoren $\boldsymbol{v}_i$ von $\boldsymbol{A}$: $\boldsymbol{x}^{(0)} = \sum_i^n c_i\boldsymbol{v}_i$
- Jede Iteration verstärkt die Komponente von $\boldsymbol{x}^{(k)}$ in Richtung $\boldsymbol{v}_1$ mehr als die anderen Komponenten

Algebraisch:

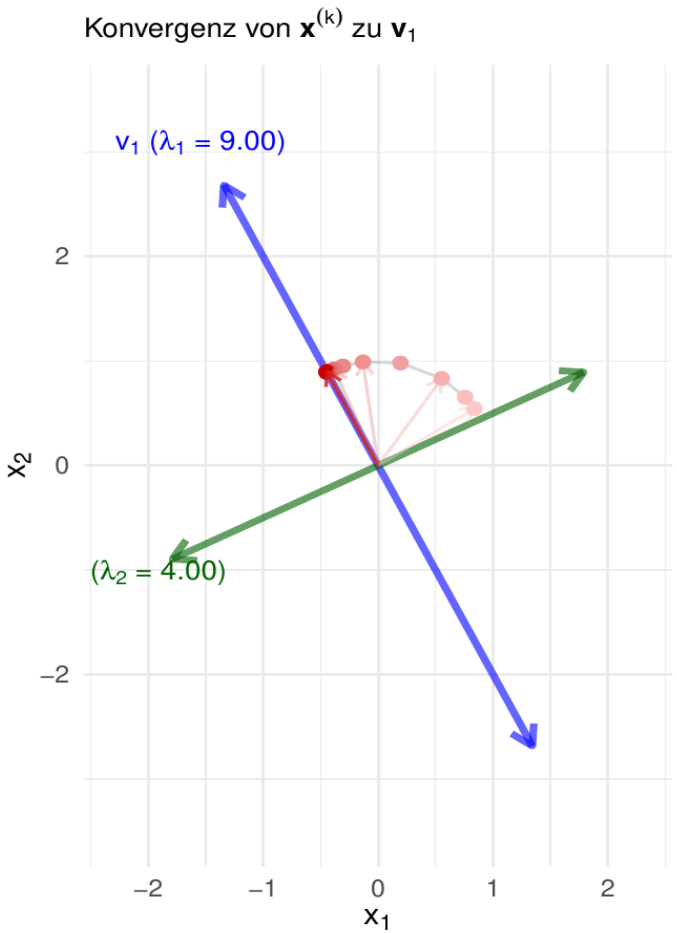
Wegen $\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v} \implies \boldsymbol{A}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$ gilt auch:

- $\boldsymbol{A}^k \boldsymbol{x}^{(0)} = \lambda_1^k \left(\boldsymbol{c}_1 \boldsymbol{v}_1 + \sum_{i=2}^{n} \boldsymbol{c}_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{v}_i \right) \approx \lambda_1^k \boldsymbol{c}_1 \boldsymbol{v}_1$ für $k \to \infty$ weil $\left(\frac{\lambda_i}{\lambda_1} \right)^k \xrightarrow{k \to \infty} 0$ für $i \geq 2$
- $\frac{\boldsymbol{A}^k \boldsymbol{x}^{(0)}}{\| \boldsymbol{A}^k \boldsymbol{x}^{(0)} \|} \approx \frac{\lambda_1^k \boldsymbol{c}_1 \boldsymbol{v}_1}{|\lambda_1^k \boldsymbol{c}_1| \| \boldsymbol{v}_1 \|} = \pm \boldsymbol{v}_1$

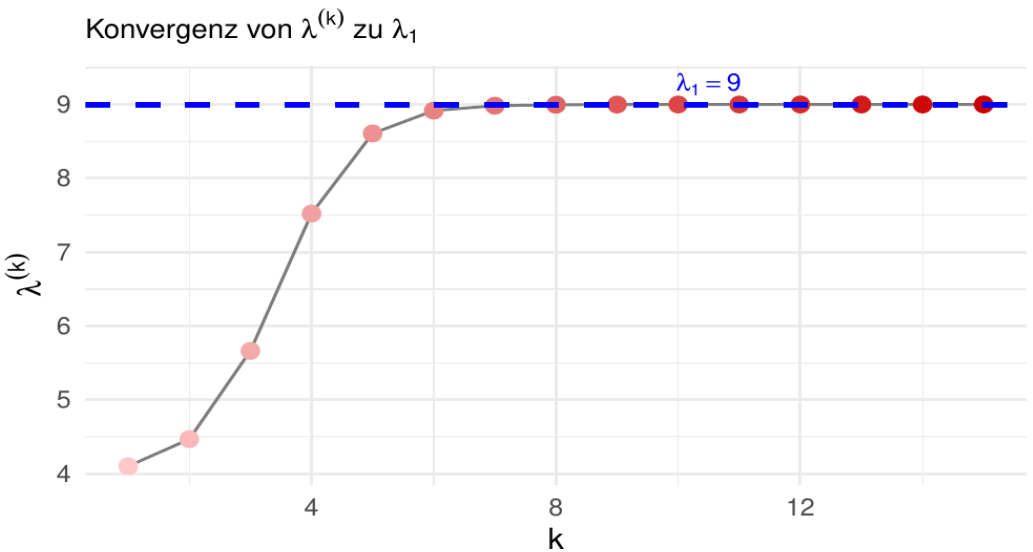
Potenzmethode: Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \boldsymbol{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \boldsymbol{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ mit } \boldsymbol{x}^{(0)} = \begin{pmatrix} 2 \\ 1.3 \end{pmatrix}$$

:



k	Winkel zu v[1]
0	83.54°
1	75.71°
2	60.18°
3	37.79°
5	8.71°
10	0.15°
15	0.00°



Quiz

Für welche $\boldsymbol{x}^{(0)}$ versagt die Potenzmethode?

1. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_1$
2. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_2$
3. $\boldsymbol{x}^{(0)} = c(\boldsymbol{v}_1 + \boldsymbol{v}_2)$

Iterative Berechnungsverfahren

- Wir wollen (alle) Eigenwerte/-vektoren von $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ finden.
- Idee: Überführe $\boldsymbol{A}$ in spezielle Matrix $\boldsymbol{B}$, so dass das Problem leichter wird.

Theorem

Sei $\boldsymbol{B}$ eine zu $\boldsymbol{A}$ ähnliche Matrix (d.h. $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$). Dann gilt:

- $\boldsymbol{A}$ und $\boldsymbol{B}$ haben die gleichen Eigenwerte (und damit auch gleiche Determinante und Spur).
- Ist $\boldsymbol{y}$ Eigenvektor von $\boldsymbol{B}$, so ist $\boldsymbol{x} = \boldsymbol{Q}^{-1}\boldsymbol{y}$ Eigenvektor von $\boldsymbol{A}$ zum gleichen Eigenwert.

Beweis:

$$\boldsymbol{B}\boldsymbol{y} = \lambda \boldsymbol{y} \implies \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}\boldsymbol{y} = \lambda \boldsymbol{y} \implies \boldsymbol{A}\boldsymbol{Q}^{-1}\boldsymbol{y} = \lambda \boldsymbol{Q}^{-1}\boldsymbol{y} \implies \boldsymbol{A}\boldsymbol{x} = \lambda \boldsymbol{x}.$$

Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \boldsymbol{Q} = \tfrac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$$

- $\boldsymbol{Q}$ ist **orthogonal**, also $\boldsymbol{Q}^{-1} = \boldsymbol{Q}^\top$: das Beispiel spezialisiert die Ähnlichkeitstransformation $\boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$ auf *orthogonale* Ähnlichkeit $\boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^\top$.

$$\boldsymbol{B} = \boldsymbol{Q}(\boldsymbol{A}\boldsymbol{Q}^\top) = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

(Rechnung → Anhang)

- $\boldsymbol{B}$ diagonal $\Longrightarrow$ Eigenwerte direkt ablesbar: 4 und 9
- **Der Punkt:**
  Ähnliche Matrizen haben dieselben Eigenwerte — $\boldsymbol{A}$ hat also ebenfalls die Eigenwerte 4 und 9, obwohl man sie an $\boldsymbol{A}$ nicht ablesen kann.

Verifikation:

$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = (5-\lambda)(8-\lambda) - 4 = \lambda^2 - 13\lambda + 36 = (\lambda - 4)(\lambda - 9) = 0 \checkmark$$

Verfahren

Zur (iterativen) Berechnung einer zu $\boldsymbol{A}$ ähnlichen Matrix $\boldsymbol{B}^{(k)}$ deren Eigenwerte einfach zu bestimmen sind gibt es verschiedenste Ideen/Methoden:

- `QR-Iteration` führt zu (approximativer) Dreiecksmatrix $\boldsymbol{B}$ (*Schur-Zerlegung*).
  - ähnlich für `LU`- und `Cholesky`-Iteration
- *Lanczos-Iteration* führt zu tridiagonalem $\boldsymbol{B}$ ($b_{ij} = 0$ für $|i - j| > 1$) und kann auf die "wichtigsten" Eigenwerte eingeschränkt werden. Effizienter für große, dünn besetzte Matrizen.
- In der Praxis brauchen wir zusätzliche numerische Tricks, um diese Methoden `stabil` und akzeptabel schnell zu machen: *Shifted QR*, *Implicitly Restarted Lanczos*

QR-Iteration

Def.: QR-Iteration

Die **QR-Iteration** nutzt die *QR-Zerlegung* zur Iteration:

1. Starte mit $\boldsymbol{A}^{(0)} = \boldsymbol{A}$
2. Für $k = 1, 2, \ldots$:
   - Berechne QR-Zerlegung: $\boldsymbol{A}^{(k-1)} = \boldsymbol{Q}^{(k)} \boldsymbol{R}^{(k)}$
   - Setze $\boldsymbol{A}^{(k)} = (\boldsymbol{Q}^{(k)})^\top \boldsymbol{A}^{(k-1)} \boldsymbol{Q}^{(k)} = \boldsymbol{R}^{(k)} \boldsymbol{Q}^{(k)}$

- $\boldsymbol{A}^{(k)}$ sind ähnlich zu $\boldsymbol{A}$:

$$\boldsymbol{A}^{(k)} = \boldsymbol{R}^{(k)} \boldsymbol{Q}^{(k)} = (\boldsymbol{Q}^{(k)})^{-1} \boldsymbol{A}^{(k-1)} \boldsymbol{Q}^{(k)} = (\boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)})^{-1} \boldsymbol{A} (\boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)})$$

- Für $k \to \infty$ konvergiert $\boldsymbol{A}^{(k)}$ gegen eine obere Dreiecksmatrix (!) $\to$ Eigenwerte auf der Diagonalen
  (für symmetrisches $\boldsymbol{A}$ mit separierten $|\lambda_i|$; allgemein: Schur-Form)
- Berechnung der Eigenvektoren $\boldsymbol{V}$ über Produkt $\boldsymbol{Q}_k := \prod_{j=1}^{k} \boldsymbol{Q}^{(j)}$ (für symmetrische $\boldsymbol{A}$).

Beweis, dass $\boldsymbol{A}^k = \boldsymbol{Q}_k \boldsymbol{R}_k$: siehe Anhang

QR-Iteration: Interpretation

- Die QR-Iteration berechnet iterativ die *QR-Zerlegung* von $\boldsymbol{A}^k$:

$$\boldsymbol{A}^k = \boldsymbol{Q}_k \boldsymbol{R}_k \text{ mit } \boldsymbol{Q}_k := \boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)}$$

  (Beweis $\rightarrow$ Anhang)

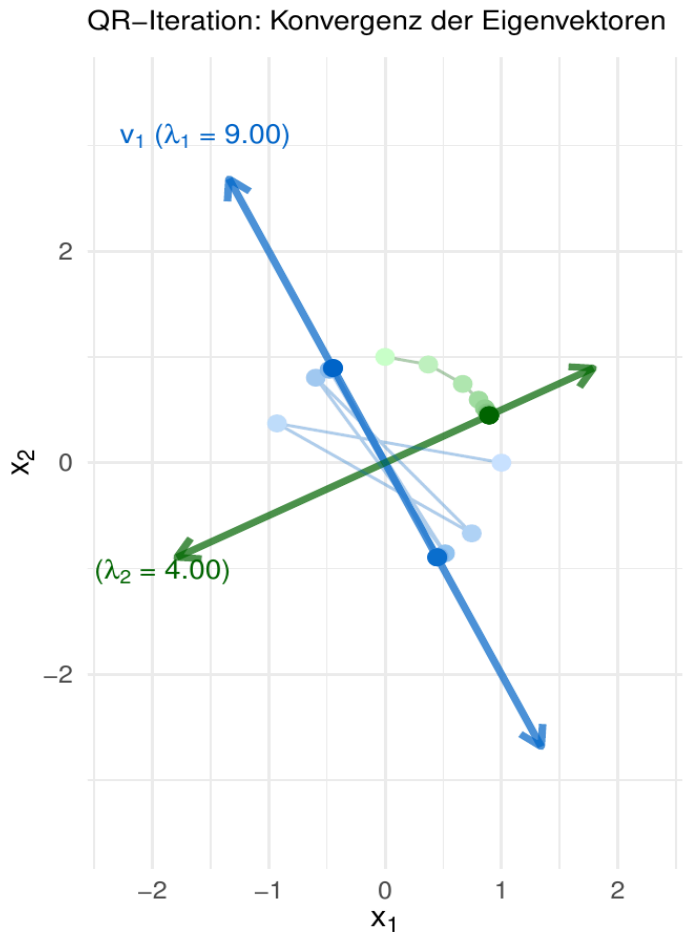
- Spalten von $\boldsymbol{Q}_k$ sind orthonormalisierte Versionen von $\boldsymbol{A}^k \boldsymbol{e}_1, \boldsymbol{A}^k \boldsymbol{e}_2, \ldots, \boldsymbol{A}^k \boldsymbol{e}_n$
  $\rightarrow$ entspricht *simultaner* *Potenzmethode* auf **allen** Einheitsvektoren!
  $\implies \boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}$

- $\boldsymbol{A}^{(k)} = \boldsymbol{Q}_k^\top \boldsymbol{A} \boldsymbol{Q}_k$ konvergiert wegen $\boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}$ gegen eine obere Dreiecksmatrix mit Eigenwerten auf der Diagonale (*Schur-Zerlegung*);
  für *symmetrische* $\boldsymbol{A} = \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^\top$ (mit *orthogonalem* $\boldsymbol{V}$) sogar gegen die Diagonalmatrix $\boldsymbol{\Lambda}$:

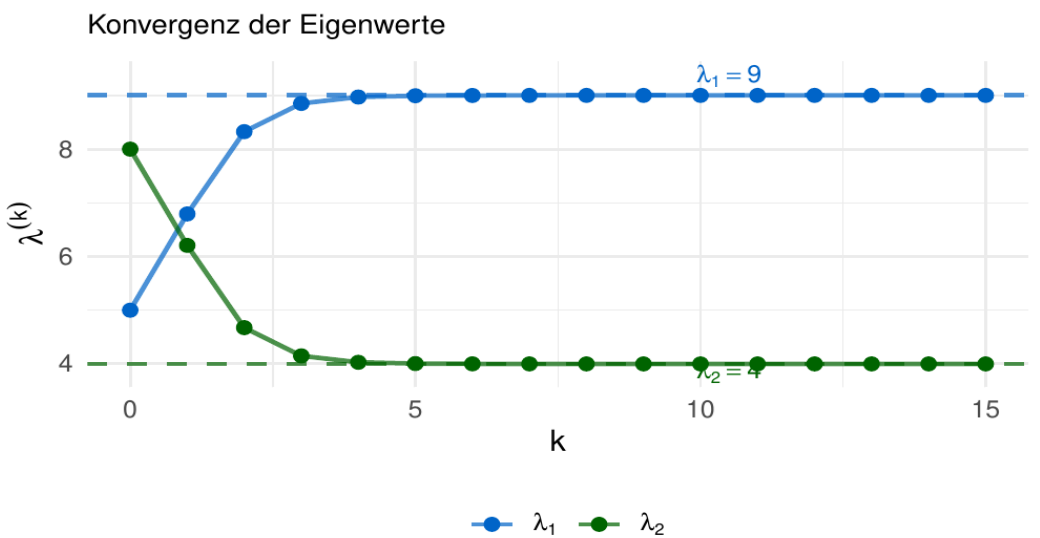
$$\boldsymbol{Q}_k^\top \boldsymbol{A} \boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}^\top \boldsymbol{A} \boldsymbol{V} = \boldsymbol{V}^\top \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^\top \boldsymbol{V} = \boldsymbol{\Lambda}$$

QR-Iteration: Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \boldsymbol{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \boldsymbol{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$



k	Winkel zu v[1]	Winkel zu v[2]
0	116.57°	116.57°
1	41.63°	138.37°
2	158.44°	158.44°
3	9.96°	170.04°
5	1.99°	178.01°
10	179.97°	179.97°
15	0.00°	180.00°



Bemerkungen

`Konditionierung` von Eigenwertproblemen:

- Eigenwertprobleme sind *gut* `konditioniert`, wenn die Matrix nahezu *symmetrisch* ist.
- Eigenvektorprobleme sind *schlecht* `konditioniert`, wenn *Eigenwerte nahe beieinander* liegen.

Laufzeit:

- Jede QR-Iteration ist $O(n^3)$
- Insgesamt werden oft nur wenige Iterationen benötigt (abhängig von Eigenwertverteilung).
- Iterative `SVD`-Verfahren (z.B. *Lanczos Bi-Diagonalization*; `{irlba}`) sind noch schneller für große, dünn besetzte Matrizen

Anwendung: Google PageRank

Problem:
Rangordnung von $n \approx 10^{10}$ Webseiten im Internet bilden.

Modell:
Jede Seite i hat einen "Wichtigkeits-Score" x_i, der die Wichtigkeit der Seiten verlinkt auf i gewichtet:

$$x_i = \sum_{j:j\to i} \frac{x_j}{d_j} \quad \Longleftrightarrow \quad \boldsymbol{x} = \boldsymbol{A}\boldsymbol{x} \text{ mit } \boldsymbol{A} = \left[\frac{I(j\to i)}{d_j}\right]_{i,j} \Longrightarrow \text{Eigenwertproblem mit } \lambda = 1!$$

("$j \to i$" heißt "Seite j verlinkt auf i", d_j sei Anzahl ausgehender Links von j)

(Setzt voraus, dass jede Seite mindestens einen ausgehenden Link hat — sonst ist $d_j = 0$ und die Spalte j gar nicht definiert; solche Spalten füllt man gleichverteilt auf.)

Frage: *Warum kann Google keine $10^{10} \times 10^{10}$-Matrix invertieren — und warum funktioniert die Eigenwert-Berechnung trotzdem?*

- Direkte Methoden unmöglich: $\boldsymbol{A}$ dicht gespeichert wären 10^{20} Einträge ($\approx$ 800 Exabyte), Inversion/Zerlegung $O(n^3) = 10^{30}$ Operationen.
- Aber: $\boldsymbol{A}$ ist *extrem dünn besetzt* (jede Seite hat nur ~ 10 Links) $\Longrightarrow$ Speicherbedarf $O(n)$ statt $O(n^2)$
- **Potenzmethode**: $\boldsymbol{x}^{(k+1)} = \boldsymbol{A}\boldsymbol{x}^{(k)} / \left\|\boldsymbol{A}\boldsymbol{x}^{(k)}\right\|$ braucht nur Matrix-Vektor-Produkte: eine Iteration ist $O(n)$, Konvergenz in ~ 50 Iterationen
- **Gesamt:** $O(n)$ statt $O(n^3)$ — macht Google überhaupt erst möglich!

Anwendung: Hauptkomponentenanalyse (PCA)

Berechnung von Eigenwerten/-vektoren für Datenanalysen essentiell:

- **Problem:** Gegeben ist eine mittelwert-zentrierte Datenmatrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ (n Beobachtungen, p Variablen)
- **Ziel:** Finde Richtungen maximaler Varianz zur Dimensionsreduktion
- **Lösung:** Berechne die Kovarianzmatrix $\boldsymbol{\Sigma} = \frac{1}{n-1} \boldsymbol{X}^\top \boldsymbol{X} \in \mathbb{R}^{p \times p}$
  - Eigenvektoren $\boldsymbol{v}_j$ von $\boldsymbol{\Sigma}$ sind die *Hauptachsen* bzw. *Loading-Richtungen* maximaler Varianz in $\mathbb{R}^p$
  - Die *Hauptkomponenten* (Scores) sind $\boldsymbol{z}_j = \boldsymbol{X}\boldsymbol{v}_j$; die Eigenwerte geben ihre Varianz an
- In der Praxis noch schneller/effizienter über *SVD* von $\boldsymbol{X}$ (z.B. `{irlba}: irlba()` (*iterativ*), `svdr()` (*iterativ* & *probabilistisch*))

Warum iterative Methoden?

- Moderne Anwendungen: $p = 10^4$ bis 10^6 (z.B. Gen-Expression, Bildanalyse, ...)
- Oft nur die k größten Eigenwerte benötigt ($k \ll p$) - vgl. Bildkompression durch *Reduzierte SVD*
- Lanczos-Iteration: Berechnet nur die k größten Eigenwerte in $O(kp^2)$ statt $O(p^3)$ für alle
  - Bei $p = 100.000$ und $k = 10$ ist Lanczos ~ 10.000 mal schneller!

Anwendung: Approximative SVD

Bildkompression durch *Reduzierte SVD*: X ist ein Graustufen-Foto als 2500×3300-Matrix *(Setup-Code, Fehlervergleich & Bildpanel → Anhang)*:

```r
r <- 50
microbenchmark::microbenchmark(
  svd_full   = svd(X),                          # exakt: alle Singulaerwerte
  svd_irlba  = irlba::irlba(X, nv = r, nu = r),  # iterativ (Lanczos): nur r
  svd_random = irlba::svdr(X, k = r),            # iterativ & randomisiert
  times = 5)
```

```
Unit: seconds
       expr   min    lq   mean median     uq    max neval cld
   svd_full 84.14 92.85 107.77 112.60 114.01 135.23     5  a
  svd_irlba  1.98  2.09   2.85   2.13   3.46   4.58     5   b
 svd_random  5.79  7.00   9.36  10.44  10.77  12.80     5   b
```

```r
X_svdr <- with(irlba::svdr(X, k = r), u %*% diag(d) %*% t(v))
norm(X - X_svdr, "F") / norm(X, "F")  # rel. Fehler der Rang-50-Approximation
```

```
[1] 0.129
```

→ iterative/randomisierte Verfahren sind hier $\sim 40\times$ bzw. $\sim 11\times$ schneller — bei praktisch identischem Ergebnis (Vergleich mit exakter Rang-50-SVD → Anhang).

Iterative Eigenwertverfahren: Vergleich

Methode	Komplexität / Iteration	Anwendung	Output
Potenz-methode	$O(\mathrm{nnz}(\boldsymbol{A}))$	Größter Eigenwert/-vektor	1 Eigenpaar
(shifted) QR	$O(n^2)$-$O(n^3)$	Alle Eigenwerte	n Eigenwerte
Lanczos	$O(m\,\mathrm{nnz}(\boldsymbol{A}) + m^2)$ für m EW	Wenige Extremal-EW, *sparse* $\boldsymbol{A}$	$m \ll n$ Eigenpaare

- Konvergenzrate *Potenzmethode*: $|\lambda_2/\lambda_1|$
- *QR-Iteration mit Shifts* oft mit quadratischer Konvergenz, am stabilsten (ohne Shifts nur linear, mit Rate $|\lambda_i/\lambda_j|$).
- Anwendung:
  - *sparse* Matrizen: Lanczos klar überlegen
  - Alle Eigenwerte, Speicher sekundär: (shifted) QR-Iteration
  - Wenige Eigenwerte: Potenzmethode (für 1), Lanczos (für $m \ll n$)

$\mathrm{nnz}(\cdot)$ = Anzahl Nicht-Null-Einträge

Iterative Lösungsverfahren

Iterative Lösungsverfahren: Intuition

- Zur Lösung von *LGS* und *KQ-Problemen* haben wir bisher direkte Methoden gesehen — *iterative Verfahren* (meist *Fixpunktiterationen*) sind oft viel schneller, wenn man etwas Genauigkeit opfert.

Wie nähern wir uns iterativ der Lösung?

- *Residuum*:
  $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}$ misst, wie stark $\boldsymbol{x}^{(k)}$ die Gleichung $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ verletzt
- Falls $\boldsymbol{A}\boldsymbol{x}^{(k)} = \boldsymbol{b}$, dann ist $\boldsymbol{r}^{(k)} = \boldsymbol{0}$ und wir haben die Lösung gefunden
- *Korrektur:*
  Solange $\boldsymbol{r}^{(k)} \neq \boldsymbol{0}$, korrigieren wir unsere Schätzung:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \underbrace{\boldsymbol{C}\boldsymbol{r}^{(k)}}_{\text{Korrekturschritt}}$$

- $\boldsymbol{C}$ bestimmt, wie stark wir wo in Richtung des Residuums korrigieren
- **Analogie:** *Gradient Descent* - wir bewegen uns Schritt für Schritt zur Lösung

Warum nicht direkt lösen?
$\rightarrow$ Bei großen und/oder dünnbesetzten Matrizen sind iterative Methoden oft um Größenordnungen schneller als direkte Verfahren!

Iterative Lösungsverfahren für LGS

- Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, $\boldsymbol{b} \in \mathbb{R}^n$ und $\boldsymbol{x}$ die Lösung des LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.
- Für einen beliebigen Startwert $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$ und $\boldsymbol{C} \in \mathbb{R}^{n \times n}$, definiere die Iteration

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \boldsymbol{C}(\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k-1)}), \quad k = 1, 2, \ldots.$$

Theorem

Sei $\boldsymbol{x}$ die eindeutige Lösung des LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

Falls $\rho := \|\boldsymbol{I}_n - \boldsymbol{C}\boldsymbol{A}\|_2 < 1$ (*Spektralnorm*), gilt für alle $k \geq 1$:

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|_2.$$

- Insbesondere: $\lim_{k \to \infty} \boldsymbol{x}^{(k)} = \boldsymbol{x}$ (*Konvergenz*).

Beweis: siehe Anhang

Quiz

Sei

$$\boldsymbol{x}^{(k)} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}$$

mit $\boldsymbol{C} = \gamma \boldsymbol{I}_n$ für ein $\gamma > 0$. Was ist die Laufzeitkomplexität eines Iterationsschrittes?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Lösung: 3. — Mit $\boldsymbol{C} = \gamma \boldsymbol{I}$ ist $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} - \gamma(\boldsymbol{A}\boldsymbol{x}^{(k-1)}) + \gamma \boldsymbol{b}$: pro Schritt eine dichte Matrix-Vektor-Multiplikation $\boldsymbol{A}\boldsymbol{x}^{(k-1)}$, also $O(n^2)$; Skalierung mit γ und die zwei Vektoradditionen sind nur $O(n)$.

Iterative Lösungsverfahren für LGS: Wahl von C

- Die Frage ist nun, wie wir $\boldsymbol{C}$ so wählen, dass:
  - $\rho = \|\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A}\|_2 < 1$ möglichst klein ist (optimal: $\boldsymbol{C} \approx \boldsymbol{A}^{-1}$),
  - $\boldsymbol{x}^{(k)} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}$ einfach zu berechnen ist.

Beliebte Wahlen für C mit Iterationsaufwand $O(n^2)$:

- *Richardson*-Iteration: $\boldsymbol{C} = \gamma \boldsymbol{I}_n$; konvergiert z.B. für SPD $\boldsymbol{A}$, falls $0 < \gamma < 2/\lambda_{\max}(\boldsymbol{A})$.
- *Jacobi*-Iteration: $\boldsymbol{C} = \operatorname{diag}(\boldsymbol{A}_{11}, \ldots, \boldsymbol{A}_{nn})^{-1}$.
- *Gauss-Seidel*-Iteration: $\boldsymbol{C}^{-1}$ gleich unterem Dreieck von $\boldsymbol{A}$.

Gesamtaufwand:

- Wegen $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|$ reichen $k = O(\log(1/\epsilon))$ Schritte für Genauigkeit ϵ (→ Self-Check).
- Damit brauchen wir $O(n^2 \log(1/\epsilon))$ Operationen für $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \epsilon$.
- Vergleiche: $O(n^3)$ für exakte Lösungen durch *LU* oder *QR*.

Analoge iterative Verfahren gibt es auch für *KQ*-Probleme; in seltenen Fällen sind sie sogar *exakt*. Meist gilt: Trade-off zwischen *Laufzeit* und Genauigkeit — steuerbar über die Iterationszahl.

Bsp: Richardson-Iteration

Gegeben: $\boldsymbol{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. Wahre Lösung: $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b} = \frac{1}{11}\begin{pmatrix} 1 \\ 7 \end{pmatrix} \approx \begin{pmatrix} 0.091 \\ 0.636 \end{pmatrix}$.

Richardson mit $\gamma = 0.25$, Startwert $\boldsymbol{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

Iteration 0: $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} + 0.25\boldsymbol{r}^{(0)} = \begin{pmatrix} 0.25 \\ 0.5 \end{pmatrix}$

Iteration 1: $\boldsymbol{A}\boldsymbol{x}^{(1)} = \begin{pmatrix} 1.5 \\ 1.75 \end{pmatrix}$, $\boldsymbol{r}^{(1)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(1)} = \begin{pmatrix} -0.5 \\ 0.25 \end{pmatrix}$, $\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} + 0.25\boldsymbol{r}^{(1)} = \begin{pmatrix} 0.125 \\ 0.563 \end{pmatrix}$

Iteration 2: $\boldsymbol{A}\boldsymbol{x}^{(2)} = \begin{pmatrix} 1.063 \\ 1.813 \end{pmatrix}$, $\boldsymbol{r}^{(2)} = \begin{pmatrix} -0.063 \\ 0.188 \end{pmatrix}$, $\boldsymbol{x}^{(3)} = \begin{pmatrix} 0.109 \\ 0.609 \end{pmatrix}$

Iteration 3: $\boldsymbol{x}^{(4)} = \begin{pmatrix} 0.098 \\ 0.625 \end{pmatrix}$

Fehler: $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2$: $0.643 \to 0.210 \to 0.081 \to 0.033 \to 0.013 \to \cdots$

Konvergenzrate: $\rho = \|\boldsymbol{I} - \gamma\boldsymbol{A}\|_2 \approx 0.41 \implies$ Fehler sollte ca. um Faktor 2.5 pro Iteration sinken.

Probabilistische Methoden

Zufall mit Absicht

- Oft sind Probleme so groß, dass wir gerne Genauigkeit für bessere *Komplexität* tauschen.
- Heute wird das zunehmend durch *Randomisierung* erreicht.
- **Hauptidee:**
  - *Ersetze* großes Problem durch ein
  - *zufällig* erzeugtes, *kleines* Problem,
  - das *höchstwahrscheinlich* eine *ähnliche Lösung* hat.

Idee: Matrix-Sketching

Geometrische Fakten in hoch-dimensionalem Raum $\mathbb{R}^n$ mit $n \gg 1$:

Für großes n sind zwei zufällige Vektoren $\boldsymbol{s}_1, \boldsymbol{s}_2 \in \mathbb{R}^n$ mit hoher Wahrscheinlichkeit fast orthogonal.

$\Longrightarrow$ geeignet *zufällig* gezogene Matrix $\boldsymbol{S} \in \mathbb{R}^{m \times n}$ mit $m \ll n$ erhält mit hoher Wahrscheinlichkeit

- Abstände: $\|\boldsymbol{S}\boldsymbol{x} - \boldsymbol{S}\boldsymbol{y}\|_2 \approx \|\boldsymbol{x} - \boldsymbol{y}\|_2$
- Winkel: $\angle(\boldsymbol{S}\boldsymbol{x}, \boldsymbol{S}\boldsymbol{y}) \approx \angle(\boldsymbol{x}, \boldsymbol{y})$

$\Longrightarrow$ Eine **zufällige Projektion** $\boldsymbol{S}$ in einen niedrigdimensionaleren Raum kann Geometrie oft näherungsweise erhalten.

Beispiel: Sketching in R

```r
set.seed(1312)
n <- 10000                # zwei beliebige n-dimensionale Vektoren:
x <- runif(n); y <- runif(n)
m <- 50                   # m << n !!
S <- rnorm(m * n, sd = sqrt(1 / m)) |> matrix(nrow = m, ncol = n)
Sx <- S %*% x;  Sy <- S %*% y
```

Wir ersetzen $n = 10000$ Koordinaten durch $m = 50$ zufällige Projektionen.

Vorhersage:

Um wie viel Prozent ändern sich Distanz $\|\boldsymbol{x} - \boldsymbol{y}\|_2$ und Winkel $\angle(\boldsymbol{x}, \boldsymbol{y})$ dabei wohl —
eher $\sim 50\%$, $\sim 3\%$ oder $< 0.1\%$?

```r
angle <- function(x, y) (sum(x * y) / (norm(x, "2") * norm(y, "2"))) |> acos()
c(norm(x - y, "2"), norm(Sx - Sy, "2"))  ## Distanzen  ||x - y||, ||Sx - Sy||
c(angle(x, y), angle(Sx, Sy))            ## Winkel (rad)
```

```
[1] 41.0 42.2
[1] 0.728 0.735
```

$\rightarrow$ nur $\approx 3\%$ [$\approx 1\%$] Abweichung in Distanz [Winkel] — obwohl 99.5% der Dimensionen wegfallen!

(Eine einzelne Ziehung, und eine recht glückliche: für Gauss-Skizzen streut die relative Abweichung der Distanz mit $\approx 1/\sqrt{2m} = 10\%$.)

Zufällige Einbettung eines festen Vektors

Theorem

Sei

- $\boldsymbol{S} \in \mathbb{R}^{m \times n}$ eine Matrix mit i.i.d. Zeilen $\boldsymbol{s}_i^\top$ (wobei $\boldsymbol{s}_i \in \mathbb{R}^n$),
- $\mathbb{E}[\boldsymbol{s}_i \boldsymbol{s}_i^\top] = \boldsymbol{I}_n / m$ und $\mathbb{E}[(\boldsymbol{s}_i^\top \boldsymbol{x})^4] \leq K \|\boldsymbol{x}\|^4 / m^2$ für alle $\boldsymbol{x} \in \mathbb{R}^n$.

Dann gilt für jeden *festen* Vektor $\boldsymbol{x} \in \mathbb{R}^n$ mit einer Wahrscheinlichkeit von mindestens $1 - \delta$

$$(1 - \epsilon)\|\boldsymbol{x}\|^2 \leq \|\boldsymbol{S}\boldsymbol{x}\|^2 \leq (1 + \epsilon)\|\boldsymbol{x}\|^2,$$

wobei $\delta > 0$ und $\epsilon = \sqrt{K / \delta m}$.

Beispiele:

- *Gauss:* $\boldsymbol{s}_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \boldsymbol{I}_n / m)$.
- *Rademacher:* s_{ij} *i.i.d.* mit $\mathbb{P}(s_{ij} = 1/\sqrt{m}) = \mathbb{P}(s_{ij} = -1/\sqrt{m}) = 1/2$.
- *Subsampling:* $\boldsymbol{s}_i = \sqrt{n/m}\, \boldsymbol{e}_{K_i}$ skalierter Einheitsvektor mit $K_i \sim \mathrm{Unif}\{1, \ldots, n\}$.

Beweis: siehe Anhang

Anwendungen

- Zufällige Einbettungen (*Matrix-Sketching*) haben zahlreiche Anwendungen:
  - Matrixmultiplikation: $\boldsymbol{AB} \approx (\boldsymbol{AS}^\top)(\boldsymbol{SB})$
  - *KQ-Probleme*: $\min \|\boldsymbol{Ax} - \boldsymbol{b}\|_2 \approx \min \|\boldsymbol{SAx} - \boldsymbol{Sb}\|_2$
  - ...
- *Achtung:* Unser Theorem gilt je *festem* Vektor — hier hängt z.B. der Minimierer selbst von $\boldsymbol{S}$ ab!
  Dafür braucht man stärkere, *simultane* Garantien (*Subspace Embeddings*): gleiche Beweisidee, etwas größeres m.
- Wirklich schneller werden Algorithmen aber nur, wenn wir Produkte $\boldsymbol{Sx}$ sehr schnell berechnen können.
- *Beispiele:*
  - Wenn $\boldsymbol{S}$ m zufällige Koordinaten aus $\boldsymbol{x}$ zieht, braucht es dafür nur $O(m)$ Operationen.
  - Besonders leistungsfähig: *Subsampled Randomized Hadamard Transform* (SRHT) braucht $O(n \log m)$ Rechenschritte. (Vergleich der Sketching-Matrizen $\rightarrow$ Anhang)

Beispiel: Dimensionsreduktion mit Matrix-Sketching

Gegeben:
$n = 10000$ Dimensionen, $\delta = 0.05$, $\epsilon = 0.1$

Frage:
Wie klein kann m sein?

Aus dem Theorem:
Mit $\boldsymbol{s}_i \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_n/m)$ gilt $K = 3$ und

$$\epsilon = \sqrt{\frac{K}{\delta m}} \quad \Longrightarrow \quad m = \frac{K}{\delta \epsilon^2} = \frac{3}{0.05 \cdot 0.01} = 6000$$

Ergebnis:

Mit $m = 6000$ Dimensionen können wir $n = 10000$ Dimensionen so darstellen, dass jeder gegebene Abstand mit 95% Wahrscheinlichkeit um höchstens 10% verzerrt wird.

Praktische Konsequenz:

- Speicher & Rechenzeit pro Distanzberechnung: $O(n) \to O(m)$
- Die Schranke (via Tschebyscheff-Ungleichung) ist sehr *konservativ* — in der Praxis reichen oft viel kleinere m: die Demo oben kam mit $m = 50$ auf nur $\approx 3\%$ Abweichung!
- Anwendung: Schnelle Nächste-Nachbar-Suche, Clusteranalyse in hohen Dimensionen, ...

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Lösungsansätze für Eigenwertprobleme
- *Iterative Verfahren* nähern sich Lösung schrittweise durch *Fixpunktiteration*
- *Probabilistische Verfahren* ersetzen schweres durch zufälliges, leichteres (niedrigdimensionaleres) Problem mit approximativer Lösung
- Beide geben uns einen Trade-off zwischen *Laufzeit* und Genauigkeit

Nächste Vorlesung

- Tensoren und Tensorprodukte

Self-Check (1/2)

Frage 1: Sei $\rho \in (0, 1)$, $\epsilon > 0$, und

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|.$$

Wie groß muss k mindestens sein, so dass $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \epsilon$?

1. $k = O(1)$
2. $k = O(\log(1/\epsilon))$
3. $k = O(1/\epsilon)$
4. $k = O(1/\epsilon^k)$

Lösung: 2. – mit $e_0 = \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|$: $\rho^k e_0 \leq \epsilon \implies \rho^k \leq \epsilon/e_0$; Logarithmieren ($0 < \rho < 1 \implies \log \rho < 0$!) gibt $k \geq \frac{\log(\epsilon) - \log(e_0)}{\log(\rho)} = \frac{\log(e_0) + \log(1/\epsilon)}{-\log(\rho)}$, also minimales $k = \left\lceil \frac{\log(e_0/\epsilon)}{-\log(\rho)} \right\rceil$ (falls $0 < \epsilon < e_0$, sonst reicht schon $k = 0$) – wächst wie $\log(1/\epsilon)$. ✓

Self-Check (2/2)

Frage 2: Worin unterscheidet sich *Subsampling* (m zufällige, skalierte Zeilen) von einer *Gauss*-Sketching-Matrix vor allem?

1. Subsampling kostet nur $O(m)$ statt $O(mn)$ (Zeit & Speicher), ist aber nur zuverlässig, wenn die Information über viele Koordinaten gestreut ist.
2. Gauss-Matrizen brauchen weniger Speicher als Subsampling.
3. Subsampling erhält alle Distanzen exakt.
4. Es gibt keinen Unterschied — beide sind optimal und gleich teuer.

Frage 3: Welcher Sketching-Typ kombiniert schnelle Anwendung ($O(n \log m)$ für $\boldsymbol{Sx}$) mit sehr guter Qualität ("State of the art")?

1. Gauss
2. Rademacher
3. Subsampling
4. Subsampled Randomized Hadamard Transform (SRHT)

Anhang

Ähnlichkeit: Beispiel — Rechnung

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \boldsymbol{Q} = \tfrac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$$

Schritt 1: Berechne $\boldsymbol{AQ}^\top$:

$$\boldsymbol{AQ}^\top = \frac{1}{\sqrt{5}} \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix}$$

Schritt 2: Berechne $\boldsymbol{B} = \boldsymbol{Q}(\boldsymbol{AQ}^\top)$:

$$\boldsymbol{B} = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 20 & 0 \\ 0 & 45 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

$\boldsymbol{B}$ diagonal $\Longrightarrow$ Eigenwerte direkt ablesbar: 4 und 9.

↩ zurück: „Beispiel”

QR-Iteration: Beweis

Definiere $\boldsymbol{Q}_k := \boldsymbol{Q}^{(1)}\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k)}$, $\boldsymbol{R}_k = \boldsymbol{R}^{(k)}\boldsymbol{R}^{(k-1)}\cdots\boldsymbol{R}^{(1)}$

Zeige: $\boldsymbol{A}^k = \boldsymbol{Q}_k\boldsymbol{R}_k$

Beweis:

- *Induktionsanfang* ($k = 1$): $\boldsymbol{A} = \boldsymbol{Q}^{(1)}\boldsymbol{R}^{(1)} = \boldsymbol{Q}_1\boldsymbol{R}_1$ ✓
- *Induktionsschritt* ($k \to k+1$): $\boldsymbol{A}^{(i)} = \boldsymbol{R}^{(i)}\boldsymbol{Q}^{(i)}$,
  $\boldsymbol{A}^{(i)} = \boldsymbol{Q}^{(i+1)}\boldsymbol{R}^{(i+1)} \implies \boldsymbol{R}^{(i)}\boldsymbol{Q}^{(i)} = \boldsymbol{Q}^{(i+1)}\boldsymbol{R}^{(i+1)}$ $(\star)$

$$
\begin{aligned}
\boldsymbol{A}^{k+1} &= \boldsymbol{A}\boldsymbol{A}^k = (\boldsymbol{Q}^{(1)}\boldsymbol{R}^{(1)})(\boldsymbol{Q}_k\boldsymbol{R}_k) \\
&= \boldsymbol{Q}^{(1)}\big(\boldsymbol{R}^{(1)}\boldsymbol{Q}^{(1)}\big)\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k)}\boldsymbol{R}^{(k)}\cdots\boldsymbol{R}^{(1)} \\
&\qquad [\text{nutze } k\text{-mal } (\star) : \boldsymbol{R}^{(1)}\boldsymbol{Q}^{(1)} = \boldsymbol{Q}^{(2)}\boldsymbol{R}^{(2)}, \ldots] \\
&= \boldsymbol{Q}^{(1)}\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k+1)}\boldsymbol{R}^{(k+1)}\cdots\boldsymbol{R}^{(1)} \\
&= \boldsymbol{Q}_{k+1}\boldsymbol{R}_{k+1} \quad ✓
\end{aligned}
$$

↩ zurück: „QR-Iteration" · ↩ zurück: „QR-Iteration: Interpretation"

Iterative LGS-Verfahren: Konvergenz — Beweis

Definiere $\boldsymbol{B = I - CA}$, sodass $\|\boldsymbol{B}\|_2 = \rho$ und

$$\boldsymbol{x}^{(k)} = \boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}, \quad k = 1, 2, \dots$$

Zunächst bemerken wir, dass $\boldsymbol{x}$ ein Fixpunkt dieser Iteration ist:

$$\boldsymbol{B}\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b} = \boldsymbol{x} - \boldsymbol{C}\underbrace{(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})}_{=0} = \boldsymbol{x}$$

also

$$\begin{aligned}
\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 &= \|\boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b} - \boldsymbol{x}\|_2 \\
&= \|\boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b} - (\boldsymbol{B}\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b})\|_2 \\
&= \|\boldsymbol{B}(\boldsymbol{x}^{(k-1)} - \boldsymbol{x})\|_2 \\
&\leq \rho\|\boldsymbol{x}^{(k-1)} - \boldsymbol{x}\|_2.
\end{aligned}$$

Iterieren wir die Ungleichung, erhalten wir

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 \leq \rho\|\boldsymbol{x}^{(k-1)} - \boldsymbol{x}\|_2 \leq \rho^2\|\boldsymbol{x}^{(k-2)} - \boldsymbol{x}\|_2 \leq \cdots \leq \rho^k\|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|_2.$$ ■

↩ zurück: „Iterative Lösungsverfahren für LGS”

Zufälliges Einbetten von Unterräumen: Beweis (1/2)

1. Für $\boldsymbol{x} = \mathbf{0}$ ist die Aussage trivial. Sei also $\boldsymbol{x} \neq \mathbf{0}$ und o.B.d.A. $\|\boldsymbol{x}\| = 1$ (andernfalls teile die Ungleichung durch $\|\boldsymbol{x}\|^2$). Zu zeigen also

$$\mathbb{P}\left(1 - \sqrt{K/\delta m} \leq \|\boldsymbol{S}\boldsymbol{x}\|^2 \leq 1 + \sqrt{K/\delta m}\right) = \mathbb{P}\left(\left|\|\boldsymbol{S}\boldsymbol{x}\|^2 - 1\right| \leq \sqrt{K/\delta m}\right) \geq 1 - \delta$$

2. Mit $\|\boldsymbol{S}\boldsymbol{x}\|^2 = \sum_{i=1}^{m} \left(\boldsymbol{s}_i^\top \boldsymbol{x}\right)^2$ gilt

$$\mathbb{E}\left[\|\boldsymbol{S}\boldsymbol{x}\|^2\right] = \sum_{i=1}^{m} \mathbb{E}\left[(\boldsymbol{s}_i^\top \boldsymbol{x})^2\right] = m\boldsymbol{x}^\top \mathbb{E}\left[\boldsymbol{s}_1 \boldsymbol{s}_1^\top\right] \boldsymbol{x} = \|\boldsymbol{x}\|^2.$$

3. Weil die Summanden unabhängig sind, gilt außerdem

$$\mathbb{V}\mathrm{ar}\left[\|\boldsymbol{S}\boldsymbol{x}\|^2\right] = \sum_{i=1}^{m} \mathbb{V}\mathrm{ar}\left[(\boldsymbol{s}_i^\top \boldsymbol{x})^2\right] \leq m\mathbb{E}\left[(\boldsymbol{s}_1^\top \boldsymbol{x})^4\right] \leq \frac{K}{m}$$

   nach der Momentannahme (mit $\|\boldsymbol{x}\| = 1$).

Zufälliges Einbetten von Unterräumen: Beweis (2/2)

(Fortsetzung)

4. Nun gibt die Tschebyscheff-Ungleichung (= Markov, angewandt auf die quadrierte Abweichung)

$$\mathbb{P}\left(\left|\|\boldsymbol{S}\boldsymbol{x}\|^2 - \mathbb{E}[\|\boldsymbol{S}\boldsymbol{x}\|^2]\right| > \epsilon\right) \leq \frac{\mathbb{V}\mathrm{ar}[\|\boldsymbol{S}\boldsymbol{x}\|^2]}{\epsilon^2} \leq \frac{m\mathbb{E}[(\boldsymbol{s}_i^\top \boldsymbol{x})^4]}{\epsilon^2} \leq \frac{K}{\epsilon^2 m}.$$

   Die Wahl $\epsilon = \sqrt{K/\delta m}$ liefert die Behauptung. ■

↩ zurück: „Zufälliges Einbetten von Unterräumen"

Approximative SVD: Setup & Fehlervergleich

Setup-Code des Benchmarks (im Foliensatz ausgeführt):

```
set.seed(1312)
library(irlba)
suppressMessages(library(magick))
img <- paste0(
  "https://upload.wikimedia.org/wikipedia/commons/7/7e/",
  "Bolzano_City_Image_-_Photo_by_Giovanni_Ussi_-_In_Black_and_White_19.jpg")
X <- (image_read(img) |>
        image_data(channels = "gray") |>
        as.numeric())[,,1]
```

Relative Frobenius-Fehler der niedrigrangigen Approximationen (der exakten Rang-50-SVD zu $\boldsymbol{X}$, und der iterativen Verfahren zur exakten Rang-50-SVD):

```
X_r <- with(svd(X, nu = r, nv = r), u %*% diag(d[1:r]) %*% t(v))
X_irlba <- with(irlba::irlba(X, nu = r, nv = r), u %*% diag(d) %*% t(v))
norm(X - X_r, "F") / norm(X, "F")
```

```
[1] 0.129
```

```
c(norm(X_r - X_irlba, "F"), norm(X_r - X_svdr, "F")) / norm(X_r, "F")
```

```
[1] 2.27e-06 1.68e-04
```

$\longrightarrow$ `irlba`/`svdr` weichen von der exakten Rang-50-SVD um Größenordnungen weniger ab, als diese vom Originalbild.

Approximative SVD: Bildvergleich

Original



SVD (r = 50)



IRLBA (r = 50)



SVDR (r = 50)



↩ zurück: „Anwendung: Approximative SVD”

Vergleich Sketching-Matrizen

Typ	Speicher	Zeit für Sx	Qualität
Gauss	$O(mn)$	$O(mn)$	Optimal
Rademacher ($\pm 1/\sqrt{m}$)	$O(mn)$	$O(mn)$	Optimal
Subsampling	$O(m)$	$O(m)$	Gut (bei über viele Koordinaten *gestreuter* Information)
SRHT (Hadamard)	$O(n)$	$O(n \log m)$	Sehr gut

Trade-offs:

- **Gauss:** Theoretisch optimal, aber teuer zu speichern und anzuwenden
- **Rademacher:** Genauso gut wie Gauss, einfacher zu generieren (nur ± 1)
- **Subsampling:** Wählt m zufällige Zeilen - extrem schnell, aber nur gut, wenn die Information über viele Koordinaten gestreut ist (schlecht, wenn sie in wenigen Koordinaten konzentriert ist)
- **SRHT:** kombiniert hohe Geschwindigkeit und gute Approximationsqualität; Speicher $O(n)$ wegen der n Vorzeichen der vorgeschalteten Diagonalmatrix (sofern man sie nicht aus einem Seed neu erzeugt)
  - *Subsampled Randomized Hadamard Transform*
  - S wird nicht explizit gespeichert, sondern durch schnelle(!) Transformationen angewendet

↩ zurück: „Anwendungen”